



Find the Duplicate Number (easy)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
- Code
 - Time complexity
 - Space complexity
- Similar Problems

Problem Statement

We are given an unsorted array containing 'n+1' numbers taken from the range 1 to 'n'. The array has only one duplicate but it can be repeated multiple times. **Find that duplicate number without using any extra space.** You are, however, allowed to modify the input array.

Example 1:

Input: [1, 4, 4, 3, 2]
Output: 4

Example 2:

Input: [2, 1, 3, 3, 5, 4]
Output: 3

Example 3:

Input: [2, 4, 1, 4, 4]
Output: 4

Try it yourself

Try solving this question here:



```
1 class FindDuplicate {
2
3     public static int findNumber(int[] nums) {
4         // TODO: Write your code here
5         return -1;
6     }
7 }
```



```
6 }  
7 }  
8
```

Test

Save

Reset



Solution

This problem follows the **Cyclic Sort** pattern and shares similarities with [Find the Missing Number](#). Following a similar approach, we will try to place each number on its correct index. Since there is only one duplicate, if while swapping the number with its index both the numbers being swapped are same, we have found our duplicate!

Code

Here is what our algorithm will look like:

Java

Python3

C++

JS

```
1 class FindDuplicate {  
2  
3     public static int findNumber(int[] nums) {  
4         int i = 0;  
5         while (i < nums.length) {  
6             if (nums[i] != i + 1) {  
7                 if (nums[i] != nums[nums[i] - 1])  
8                     swap(nums, i, nums[i] - 1);  
9                 else // we have found the duplicate  
10                    return nums[i];  
11             } else {  
12                 i++;  
13             }  
14         }  
15  
16         return -1;  
17     }  
18  
19     private static void swap(int[] arr, int i, int j) {  
20         int temp = arr[i];  
21         arr[i] = arr[j];  
22         arr[j] = temp;  
23     }  
24  
25     public static void main(String[] args) {  
26         System.out.println(FindDuplicate.findNumber(new int[] { 1, 4, 4, 3, 2 }));  
27         System.out.println(FindDuplicate.findNumber(new int[] { 2, 1, 3, 3, 5, 4 }));  
28         System.out.println(FindDuplicate.findNumber(new int[] { 2, 4, 1, 4, 4 }));  
29     }  
30 }  
31
```

Run

Save

Reset



Time complexity

The time complexity of the above algorithm is $O(n)$.

Space complexity

The algorithm runs in constant space $O(1)$ but modifies the input array.



Similar Problems

Problem 1: Can we solve the above problem in $O(1)$ space and without modifying the input array?

Solution: While doing the cyclic sort, we realized that the array will have a cycle due to the duplicate number and that the start of the cycle will always point to the duplicate number. This means that we can use the fast & the slow pointer method to find the duplicate number or the start of the cycle similar to [Start of LinkedList Cycle](#).

Java

Python3

C++

JS

```
class DuplicateNumber {  
  
    public static int findDuplicate(int[] arr) {  
        int slow = 0, fast = 0;  
        do {  
            slow = arr[slow];  
            fast = arr[arr[fast]];  
        } while (slow != fast);  
  
        // find cycle length  
        int current = arr[slow];  
        int cycleLength = 0;  
        do {  
            current = arr[current];  
            cycleLength++;  
        } while (current != arr[slow]);  
  
        return findStart(arr, cycleLength);  
    }  
  
    private static int findStart(int[] arr, int cycleLength) {  
        int pointer1 = arr[0], pointer2 = arr[0];  
        // move pointer2 ahead 'cycleLength' steps  
        while (cycleLength > 0) {  
            pointer2 = arr[pointer2];  
            cycleLength--;  
        }  
        while (pointer1 != pointer2) {  
            pointer1 = arr[pointer1];  
            pointer2 = arr[pointer2];  
        }  
        return pointer1;  
    }  
}
```

Run

Save

Reset

The time complexity of the above algorithm is $O(n)$ and the space complexity is $O(1)$.

[← Back](#)[Find all Missing Numbers \(easy\)](#)[✓ Completed](#)[Next →](#)[Find all Duplicate Numbers \(€ Tr](#)

